

Teori Convolutional Neural Network

Ryan Fadhilah Faizal Hakim¹

¹Sains Data, Fakultas Sains dan Teknologi, IKOPIN University

1 Pendahuluan

Convolutional Neural Network (CNN) adalah salah satu arsitektur utama dalam *deep learning* yang dirancang untuk memproses data berbentuk grid, terutama citra digital. Data citra memiliki struktur spasial: piksel yang berdekatan biasanya memiliki hubungan semantik yang kuat. CNN memanfaatkan struktur ini melalui operasi konvolusi, yaitu operasi lokal yang menggunakan kernel atau filter untuk mengekstraksi pola seperti tepi, tekstur, bentuk, hingga objek kompleks.

Berbeda dengan *fully connected neural network* yang menghubungkan setiap neuron dengan seluruh input, CNN menggunakan koneksi lokal dan berbagi parameter (*parameter sharing*). Hal ini membuat CNN jauh lebih efisien untuk data berukuran besar seperti gambar. Jika gambar berukuran $H \times W \times C$, maka jaringan fully connected langsung terhadap seluruh piksel membutuhkan jumlah parameter sangat besar, sedangkan CNN hanya membutuhkan sejumlah kecil parameter pada setiap filter yang digunakan berulang di seluruh lokasi spasial.

Secara umum, CNN terdiri atas beberapa komponen penting:

1. lapisan konvolusi (*convolutional layer*),
2. fungsi aktivasi nonlinier,
3. lapisan pooling atau subsampling,
4. normalisasi dan regularisasi,
5. lapisan fully connected atau global pooling,
6. fungsi loss dan algoritma optimisasi.

2 Representasi Citra sebagai Tensor

Citra digital dapat direpresentasikan sebagai tensor. Untuk citra grayscale, input dapat ditulis sebagai matriks

$$\mathbf{X} \in \mathbb{R}^{H \times W}, \quad (1)$$

dengan H adalah tinggi gambar dan W adalah lebar gambar. Untuk citra berwarna RGB, input berbentuk tensor tiga dimensi

$$\mathbf{X} \in \mathbb{R}^{H \times W \times C}, \quad (2)$$

dengan $C = 3$ untuk kanal merah, hijau, dan biru. Dalam pelatihan berbasis mini-batch, sekumpulan N gambar biasanya ditulis sebagai tensor empat dimensi

$$\mathbf{X} \in \mathbb{R}^{N \times H \times W \times C} \quad (3)$$

atau dalam beberapa pustaka sebagai

$$\mathbf{X} \in \mathbb{R}^{N \times C \times H \times W}. \quad (4)$$

CNN memproses tensor tersebut secara hierarkis. Lapisan awal biasanya mempelajari fitur sederhana seperti garis dan tepi, lapisan menengah mempelajari motif dan bagian objek, sedangkan lapisan dalam mempelajari representasi semantik yang lebih abstrak.

3 Operasi Konvolusi Diskrit

Konvolusi adalah operasi inti pada CNN. Untuk sinyal satu dimensi, konvolusi diskrit antara input x dan kernel w didefinisikan sebagai

$$(x * w)[i] = \sum_{m=-\infty}^{\infty} x[m]w[i - m]. \quad (5)$$

Pada praktik deep learning, operasi yang sering digunakan sebenarnya adalah *cross-correlation*, yaitu

$$y[i] = \sum_m x[i + m]w[m], \quad (6)$$

karena kernel tidak dibalik. Namun, istilah konvolusi tetap digunakan secara luas.

Untuk citra dua dimensi dengan input $\mathbf{X}$ dan kernel $\mathbf{K}$ berukuran $k_h \times k_w$, operasi konvolusi/cross-correlation dapat ditulis sebagai

$$\mathbf{Y}[i, j] = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \mathbf{K}[u, v] \mathbf{X}[i + u, j + v]. \quad (7)$$

Jika input memiliki banyak kanal, misalnya C kanal, maka satu filter memiliki ukuran $k_h \times k_w \times C$ dan menghasilkan satu feature map:

$$\mathbf{Y}[i, j] = b + \sum_{c=1}^C \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \mathbf{K}[u, v, c] \mathbf{X}[i + u, j + v, c], \quad (8)$$

dengan b adalah bias.

Jika terdapat F filter, keluaran lapisan konvolusi menjadi tensor

$$\mathbf{Y} \in \mathbb{R}^{H_{out} \times W_{out} \times F}. \quad (9)$$

Untuk filter ke- f , persamaannya adalah

$$\mathbf{Y}[i, j, f] = b_f + \sum_{c=1}^C \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \mathbf{K}_f[u, v, c] \mathbf{X}[i+u, j+v, c]. \quad (10)$$

4 Stride, Padding, dan Ukuran Output

Dua parameter penting dalam konvolusi adalah *stride* dan *padding*. Stride s menentukan langkah pergeseran kernel. Padding p menambahkan piksel di sekitar input, biasanya bernilai nol, agar ukuran spasial keluaran dapat dikontrol.

Untuk input berukuran $H \times W$, kernel berukuran $k_h \times k_w$, padding p_h, p_w , dan stride s_h, s_w , ukuran output adalah

$$H_{out} = \left\lfloor \frac{H + 2p_h - k_h}{s_h} \right\rfloor + 1, \quad (11)$$

$$W_{out} = \left\lfloor \frac{W + 2p_w - k_w}{s_w} \right\rfloor + 1. \quad (12)$$

Jika menggunakan padding “valid”, maka $p_h = p_w = 0$ sehingga ukuran output mengecil. Jika menggunakan padding “same” dengan stride satu, padding dipilih agar

$$H_{out} = H, \quad W_{out} = W. \quad (13)$$

Untuk kernel ganjil k , padding yang umum digunakan adalah

$$p = \frac{k-1}{2}. \quad (14)$$

5 Parameter Sharing dan Local Receptive Field

CNN memiliki dua prinsip penting: *local receptive field* dan *parameter sharing*. Local receptive field berarti setiap neuron pada feature map hanya melihat wilayah kecil dari input, bukan seluruh input. Jika kernel berukuran 3×3 , maka satu nilai output hanya dipengaruhi oleh area lokal 3×3 pada input.

Parameter sharing berarti kernel yang sama digunakan di seluruh lokasi spasial. Jika satu filter berukuran $k_h \times k_w \times C$, jumlah parameternya adalah

$$k_h k_w C + 1, \quad (15)$$

dengan tambahan satu bias. Jika terdapat F filter, jumlah parameter lapisan konvolusi adalah

$$F(k_h k_w C + 1). \quad (16)$$

Jumlah ini tidak bergantung langsung pada ukuran tinggi dan lebar input, sehingga CNN efisien untuk citra besar.

Sebagai perbandingan, jika input berukuran $H \times W \times C$ dihubungkan ke M neuron fully connected, jumlah parameter adalah

$$M(HWC + 1), \quad (17)$$

yang dapat menjadi sangat besar ketika H dan W meningkat.

6 Feature Map dan Hierarki Fitur

Keluaran dari filter disebut *feature map*. Setiap filter mempelajari pola tertentu. Pada lapisan awal, filter dapat merespons tepi horizontal, tepi vertikal, atau perubahan intensitas. Pada lapisan yang lebih dalam, feature map dapat merepresentasikan pola yang lebih kompleks.

Jika lapisan ke- ℓ menerima input $\mathbf{A}^{(\ell-1)}$, maka operasi konvolusi dapat ditulis sebagai

$$\mathbf{Z}^{(\ell)} = \mathbf{K}^{(\ell)} * \mathbf{A}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad (18)$$

dan aktivasi keluarannya adalah

$$\mathbf{A}^{(\ell)} = \phi(\mathbf{Z}^{(\ell)}). \quad (19)$$

Dengan menumpuk banyak lapisan, CNN membentuk komposisi fungsi nonlinier

$$f(\mathbf{X}) = f_L \circ f_{L-1} \circ \dots \circ f_1(\mathbf{X}), \quad (20)$$

yang memungkinkan pembelajaran representasi kompleks secara hierarkis.

7 Fungsi Aktivasi

Setelah konvolusi, CNN biasanya menggunakan fungsi aktivasi nonlinier. Fungsi paling umum adalah ReLU:

$$\text{ReLU}(z) = \max(0, z). \quad (21)$$

Turunannya adalah

$$\frac{d}{dz} \text{ReLU}(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases} \quad (22)$$

ReLU membantu mengurangi masalah vanishing gradient dan mempercepat pelatihan.

Beberapa variasi ReLU antara lain Leaky ReLU:

$$\text{LeakyReLU}(z) = \max(\alpha z, z), \quad 0 < \alpha < 1, \quad (23)$$

dan ELU:

$$\text{ELU}(z) = \begin{cases} z, & z > 0, \\ \alpha(e^z - 1), & z \leq 0. \end{cases} \quad (24)$$

8 Pooling Layer

Pooling adalah operasi untuk mereduksi dimensi spasial feature map. Tujuannya adalah mengurangi jumlah komputasi, memperluas receptive field efektif, dan membuat representasi lebih toleran terhadap translasi kecil.

Pada max pooling, output adalah nilai maksimum dalam suatu jendela lokal:

$$\mathbf{Y}[i, j, c] = \max_{0 \leq u < k_h, 0 \leq v < k_w} \mathbf{X}[is_h + u, js_w + v, c]. \quad (25)$$

Pada average pooling, output adalah rata-rata:

$$\mathbf{Y}[i, j, c] = \frac{1}{k_h k_w} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \mathbf{X}[is_h + u, js_w + v, c]. \quad (26)$$

Selain pooling lokal, banyak arsitektur modern menggunakan *global average pooling* sebelum klasifikasi. Untuk feature map $\mathbf{A} \in \mathbb{R}^{H \times W \times C}$, global average pooling menghasilkan vektor

$$z_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W \mathbf{A}[i, j, c], \quad c = 1, \dots, C. \quad (27)$$

9 Fully Connected Layer dan Klasifikasi

Setelah beberapa lapisan konvolusi dan pooling, CNN dapat menggunakan lapisan fully connected untuk menghasilkan prediksi. Jika representasi akhir diratakan menjadi vektor $\mathbf{h}$, maka logit kelas dihitung sebagai

$$\mathbf{o} = \mathbf{W}\mathbf{h} + \mathbf{b}. \quad (28)$$

Untuk klasifikasi multi-kelas, probabilitas kelas ke- k diberikan oleh softmax:

$$\hat{y}_k = \frac{\exp(o_k)}{\sum_{j=1}^K \exp(o_j)}, \quad k = 1, 2, \dots, K. \quad (29)$$

Prediksi kelas adalah

$$\hat{c} = \arg \max_k \hat{y}_k. \quad (30)$$

10 Fungsi Loss

Untuk klasifikasi multi-kelas dengan label one-hot $\mathbf{y}$, fungsi loss yang umum adalah categorical cross-entropy:

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log(\hat{y}_k). \quad (31)$$

Jika label benar adalah kelas c , maka loss dapat disederhanakan menjadi

$$\mathcal{L} = -\log(\hat{y}_c). \quad (32)$$

Untuk N sampel pelatihan, objektif empirisnya adalah

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}), \quad (33)$$

dengan θ menyatakan seluruh parameter CNN.

Untuk regresi, loss yang umum digunakan adalah mean squared error:

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^N \left\| \mathbf{y}^{(n)} - \hat{\mathbf{y}}^{(n)} \right\|_2^2. \quad (34)$$

11 Backpropagation pada CNN

CNN dilatih menggunakan backpropagation dan optimisasi berbasis gradien. Untuk suatu lapisan konvolusi, misalkan

$$\mathbf{Z}[i, j, f] = b_f + \sum_{c=1}^C \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \mathbf{K}_f[u, v, c] \mathbf{X}[i+u, j+v, c]. \quad (35)$$

Jika $\delta[i, j, f] = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}[i, j, f]}$, maka gradien terhadap bobot kernel adalah

$$\frac{\partial \mathcal{L}}{\partial \mathbf{K}_f[u, v, c]} = \sum_{i, j} \delta[i, j, f] \mathbf{X}[i+u, j+v, c]. \quad (36)$$

Gradien terhadap bias adalah

$$\frac{\partial \mathcal{L}}{\partial b_f} = \sum_{i, j} \delta[i, j, f]. \quad (37)$$

Gradien terhadap input diperoleh dengan menjumlahkan kontribusi dari semua lokasi output dan semua filter yang menggunakan elemen input tersebut. Secara konseptual,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \delta * \text{rot180}(\mathbf{K}), \quad (38)$$

dengan $\text{rot180}(\cdot)$ menyatakan rotasi kernel sebesar 180° pada formulasi konvolusi matematis.

Untuk ReLU, gradien lokalnya adalah

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} \mathbb{I}(z > 0), \quad (39)$$

dengan $a = \text{ReLU}(z)$ dan $\mathbb{I}(\cdot)$ adalah fungsi indikator.

12 Optimisasi Parameter

Parameter CNN diperbarui menggunakan algoritma optimisasi. Pada stochastic gradient descent (SGD), pembaruan parameter adalah

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{J}(\theta_t), \quad (40)$$

dengan η adalah learning rate. Dengan momentum, pembaruan menjadi

$$\mathbf{v}_{t+1} = \mu \mathbf{v}_t - \eta \nabla_{\theta} \mathcal{J}(\theta_t), \quad (41)$$

$$\theta_{t+1} = \theta_t + \mathbf{v}_{t+1}, \quad (42)$$

dengan μ adalah koefisien momentum.

Optimizer Adam menggunakan estimasi momen pertama dan kedua:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (43)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad (44)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad (45)$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad (46)$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \quad (47)$$

13 Normalisasi dan Regularisasi

CNN yang dalam membutuhkan teknik stabilisasi dan regularisasi. Salah satu teknik penting adalah batch normalization. Untuk aktivasi mini-batch $x_1, x_2, \dots, x_m$, batch normalization menghitung

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \quad (48)$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2, \quad (49)$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad (50)$$

$$y_i = \gamma \hat{x}_i + \beta. \quad (51)$$

Parameter γ dan β dipelajari agar jaringan tetap dapat mengatur skala dan pergeseran aktivasi.

Regularisasi lain yang umum digunakan adalah dropout. Jika $\mathbf{h}$ adalah aktivasi dan $\mathbf{m}$ adalah mask Bernoulli, maka

$$\tilde{\mathbf{h}} = \frac{\mathbf{m} \odot \mathbf{h}}{q}, \quad m_i \sim \text{Bernoulli}(q), \quad (52)$$

dengan q adalah probabilitas unit dipertahankan.

Weight decay atau regularisasi L2 menambahkan penalti terhadap norma bobot:

$$\mathcal{J}_{\text{reg}}(\theta) = \mathcal{J}(\theta) + \lambda \|\theta\|_2^2. \quad (53)$$

14 Receptive Field Efektif

Receptive field adalah wilayah input asli yang memengaruhi suatu unit pada lapisan tertentu. Jika beberapa lapisan konvolusi ditumpuk, receptive field semakin besar. Untuk lapisan ke- ℓ dengan ukuran kernel k_ℓ dan stride s_ℓ , receptive field dapat dihitung rekursif:

$$r_1 = k_1, \quad (54)$$

$$r_\ell = r_{\ell-1} + (k_\ell - 1) \prod_{m=1}^{\ell-1} s_m. \quad (55)$$

Jika semua stride bernilai satu dan semua kernel berukuran 3, maka tiga lapisan konvolusi memiliki receptive field

$$r_3 = 3 + (3 - 1) + (3 - 1) = 7. \quad (56)$$

Artinya, tiga konvolusi 3×3 berturut-turut dapat melihat area 7×7 , tetapi dengan lebih banyak nonlinieritas dan parameter yang lebih efisien dibandingkan satu konvolusi 7×7 .

15 Varian Operasi Konvolusi

15.1 Konvolusi 1×1

Konvolusi 1×1 digunakan untuk mencampur informasi antar kanal tanpa mengubah ukuran spasial. Jika input memiliki C kanal dan output memiliki F kanal, maka

$$\mathbf{Y}[i, j, f] = b_f + \sum_{c=1}^C \mathbf{K}_f[1, 1, c] \mathbf{X}[i, j, c]. \quad (57)$$

Operasi ini sering digunakan untuk reduksi dimensi kanal dan meningkatkan efisiensi komputasi.

15.2 Dilated Convolution

Dilated convolution memperbesar receptive field tanpa menambah jumlah parameter secara signifikan. Dengan laju dilasi d , operasi dua dimensi menjadi

$$\mathbf{Y}[i, j] = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \mathbf{K}[u, v] \mathbf{X}[i + du, j + dv]. \quad (58)$$

Jika $d = 1$, operasi ini sama dengan konvolusi biasa.

15.3 Depthwise Separable Convolution

Depthwise separable convolution memisahkan konvolusi spasial per kanal dan pencampuran kanal. Pada depthwise convolution,

$$\mathbf{Y}[i, j, c] = \sum_{u, v} \mathbf{K}_c[u, v] \mathbf{X}[i + u, j + v, c]. \quad (59)$$

Kemudian pointwise convolution 1×1 mencampur kanal:

$$\mathbf{Z}[i, j, f] = \sum_{c=1}^C \mathbf{P}[1, 1, c, f] \mathbf{Y}[i, j, c]. \quad (60)$$

Jumlah parameter konvolusi biasa adalah $k_h k_w CF$, sedangkan depthwise separable convolution memiliki parameter

$$k_h k_w C + CF, \quad (61)$$

yang biasanya jauh lebih kecil.

15.4 Transposed Convolution

Transposed convolution digunakan untuk meningkatkan resolusi spasial, misalnya pada segmentasi citra dan generative model. Jika konvolusi biasa memetakan input ke output yang lebih kecil, transposed convolution mempelajari pemetaan ke ukuran yang lebih besar. Ukuran output satu dimensi sering ditulis sebagai

$$H_{out} = (H_{in} - 1)s - 2p + k + o_p, \quad (62)$$

dengan s stride, p padding, k ukuran kernel, dan o_p output padding.

16 Arsitektur CNN Populer

Beberapa arsitektur CNN berpengaruh adalah:

1. **LeNet**: arsitektur awal untuk pengenalan digit tulisan tangan.
2. **AlexNet**: memperkenalkan CNN dalam skala besar untuk klasifikasi ImageNet.
3. **VGG**: menggunakan banyak konvolusi kecil 3×3 secara bertumpuk.
4. **Inception/GoogLeNet**: menggunakan modul multi-cabang dengan beberapa ukuran kernel.
5. **ResNet**: memperkenalkan residual connection untuk melatih jaringan sangat dalam.
6. **DenseNet**: menghubungkan setiap lapisan dengan banyak lapisan sebelumnya.
7. **MobileNet**: menggunakan depthwise separable convolution untuk efisiensi perangkat mobile.

17 Residual Connection

Residual connection adalah teknik penting pada CNN modern. Alih-alih mempelajari pemetaan langsung $H(\mathbf{x})$, blok residual mempelajari fungsi residual

$$F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}. \quad (63)$$

Output blok residual adalah

$$\mathbf{y} = F(\mathbf{x}; \theta) + \mathbf{x}. \quad (64)$$

Jika dimensi input dan output berbeda, digunakan proyeksi linear:

$$\mathbf{y} = F(\mathbf{x}; \theta) + \mathbf{W}_s \mathbf{x}. \quad (65)$$

Residual connection membantu aliran gradien karena terdapat jalur identitas langsung. Jika loss adalah $\mathcal{L}$, maka

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(\frac{\partial F}{\partial \mathbf{x}} + \mathbf{I} \right). \quad (66)$$

Term $\mathbf{I}$ membantu mengurangi masalah vanishing gradient pada jaringan yang sangat dalam.

18 Data Augmentation

Data augmentation adalah teknik memperbanyak variasi data pelatihan melalui transformasi yang tidak mengubah label. Untuk citra, transformasi umum meliputi rotasi kecil, translasi, flipping, cropping, perubahan kecerahan, perubahan kontras, dan penambahan noise.

Secara konseptual, jika T adalah transformasi acak dari distribusi $\mathcal{T}$, maka model dilatih pada

$$\tilde{\mathbf{x}} = T(\mathbf{x}), \quad T \sim \mathcal{T}. \quad (67)$$

Objektif pelatihan menjadi

$$\min_{\theta} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \mathbb{E}_{T \sim \mathcal{T}} [\mathcal{L}(f_{\theta}(T(\mathbf{x})), y)]. \quad (68)$$

Dengan demikian, model didorong untuk lebih invariant terhadap transformasi yang wajar.

19 Contoh Perhitungan Konvolusi Sederhana

Misalkan input dan kernel adalah

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 1 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 \\ -1 & 2 \end{bmatrix}. \quad (69)$$

Dengan stride satu dan tanpa padding, ukuran output adalah 2×2 . Elemen pertama output adalah

$$\mathbf{Y}[1, 1] = 1(1) + 0(2) + (-1)(0) + 2(1) = 3. \quad (70)$$

Elemen lainnya adalah

$$\mathbf{Y}[1, 2] = 1(2) + 0(0) + (-1)(1) + 2(3) = 7, \quad (71)$$

$$\mathbf{Y}[2, 1] = 1(0) + 0(1) + (-1)(2) + 2(1) = 0, \quad (72)$$

$$\mathbf{Y}[2, 2] = 1(1) + 0(3) + (-1)(1) + 2(1) = 2. \quad (73)$$

Jadi hasilnya adalah

$$\mathbf{Y} = \begin{bmatrix} 3 & 7 \\ 0 & 2 \end{bmatrix}. \quad (74)$$

20 Kelebihan dan Keterbatasan CNN

Kelebihan CNN meliputi:

- efisien untuk data grid seperti citra karena parameter sharing,
- mampu mengekstraksi fitur lokal secara otomatis,
- menghasilkan representasi hierarkis dari fitur sederhana ke fitur kompleks,
- lebih tahan terhadap translasi kecil dibanding jaringan fully connected,
- dapat diperluas untuk klasifikasi, deteksi objek, segmentasi, dan generasi citra.

Namun, CNN juga memiliki keterbatasan:

- membutuhkan data besar agar generalisasi baik,
- dapat sensitif terhadap rotasi, skala, dan perubahan sudut pandang jika tidak dilatih dengan augmentasi yang sesuai,
- interpretabilitas fitur tidak selalu mudah,
- komputasi dapat mahal untuk arsitektur sangat dalam,
- kurang menangkap hubungan global jika hanya menggunakan kernel lokal tanpa mekanisme tambahan.

21 Perbandingan CNN dengan Fully Connected Network

Aspek	Fully Connected Network	Convolutional Neural Network
Struktur koneksi	Semua neuron terhubung ke semua input	Koneksi lokal melalui kernel
Jumlah parameter	Sangat besar untuk citra	Lebih efisien karena parameter sharing
Informasi spasial	Mudah hilang saat flattening	Dipertahankan sepanjang feature map
Fitur	Tidak eksplisit memanfaatkan lokalitas	Mengekstraksi fitur lokal hierarkis
Ketahanan translasi	Rendah	Lebih baik melalui konvolusi dan pooling
Aplikasi utama	Data tabular atau vektor kecil	Citra, video, sinyal grid, spektrogram

22 Aplikasi CNN

CNN digunakan dalam banyak bidang, antara lain:

1. **Klasifikasi citra:** menentukan kelas utama suatu gambar.
2. **Deteksi objek:** menemukan lokasi dan kelas objek dalam citra.
3. **Segmentasi semantik:** memberi label kelas pada setiap piksel.
4. **Pengenalan wajah:** mengekstraksi fitur wajah untuk identifikasi atau verifikasi.
5. **Analisis medis:** mendeteksi tumor, lesi, atau kelainan pada citra medis.
6. **Kendaraan otonom:** mengenali jalan, marka, rambu, pejalan kaki, dan kendaraan lain.
7. **Pemrosesan suara:** memproses spektrogram untuk klasifikasi audio atau pengenalan ucapan.
8. **Penginderaan jauh:** klasifikasi tutupan lahan, deteksi bangunan, dan pemetaan wilayah.

23 Kesimpulan

Convolutional Neural Network adalah arsitektur deep learning yang sangat efektif untuk data berstruktur grid, terutama citra. Kekuatan CNN berasal dari operasi konvolusi yang memanfaatkan local receptive field, parameter sharing, dan hierarki fitur. Secara matematis, CNN membentuk komposisi operasi linear lokal, fungsi aktivasi nonlinier, pooling, normalisasi, dan lapisan klasifikasi.

CNN lebih efisien daripada jaringan fully connected untuk citra karena jumlah parameternya jauh lebih kecil dan struktur spasial data tetap dipertahankan. Meskipun demikian, performa CNN

sangat dipengaruhi oleh desain arsitektur, ukuran kernel, stride, padding, regularisasi, augmentasi data, serta metode optimisasi. Varian modern seperti residual connection, depthwise separable convolution, dilated convolution, dan global average pooling memperluas kemampuan CNN agar lebih dalam, efisien, dan akurat untuk berbagai aplikasi dunia nyata.